offer features like CSRF protection and session state, are created in the **`routes/web.php`** file and belong to the web middleware group. The **`routes/api.php`** file defines stateless routes for the API, which are a part of the **`api`** middleware group.

- Then, the chapter explores router methods, which let programmers register routes that react to different HTTP verbs. When creating several routes with the same URI, it lists a number of common router methods and stresses the order in which they should be declared.
- The View section opens with an introduction to Laravel views and a discussion of how they help to render a web page's HTML representation. It highlights how views enable developers to segregate the display logic from the business logic of the application, resulting in simpler and more readable code.
- The first subject discussed in this part is creating views. Views in Laravel are, by default, kept in the resources/views directory. It covers how to reference views in routes and controllers and goes through the process of generating a view file.
- The chapter then examines the various view types that Laravel offers. The default templating engine in Laravel, Blade Views, is discussed. It offers capabilities like conditional statements and loops, and includes reusable code snippets.
- For developers who favor working with plain PHP code, the chapter also highlights the availability of plain PHP views. How to send data from controllers to views is covered in the section on data and views. It explains how to provide data to views using the method and the compact function, enabling developers to display dynamic content based on the supplied data.
- Then, as a means of transferring data across different views or carrying out operations on them, view composers and creators are provided. View composers are callbacks that are executed when particular views are produced, allowing data to be shared with those views. View creators, on the other hand, let programmers take action before a view is produced.